

Introduction to Programming
COL 1000 Practice sheets

Department of Computer Science & Engineering
IIT Delhi

Contents

1	Introduction and Python Basics	3
2	Conditionals: If-Else	7
3	While Loops	11
4	For Loops, Break Statements, and Nested Loops	14
5	Strings and String Slicing	18
6	Functions and Basic Recursion	21
7	Binary Search on Integers and Strings	24
8	Lists and Tuples	28
9	Recursions using Lists and Tuples	33
10	Aliasing and References	39
11	Invariants	44
12	Dictionaries (Hash Maps)	49
13	Time Complexity	53
14	Classes and Inheritance	56
15	File Handling	64
16	Try and Except	67

Chapter 1

Introduction and Python Basics

Output Prediction

Predict the output or fix the errors for each of the following programs.

1.

```
print("Hello")  
print("Python")
```

2.

```
x = 13  
y = 3  
print(x + y)  
print(x * y)  
print(x % y)
```

3.

```
name = "Rama"  
print(name * 3)  
print(name + 3)
```

4.

```
a = 10  
b = 3  
print(a / b)  
print(a // b)  
print(a % b)
```

5.

```
s1 = "Hello"  
s2 = "World"  
print(s1 + " " + s2)
```

6.

```
print("5" + "6")  
print(5 + 6)
```

7.

```
x = 4
x = x//2 + 1
x = x * 2
print(x)
```

8.

```
x = "\nPython"
x = x * 2
print(x+" is a programming language.")
```

9.

```
val_x = 5
val_y = "5"
print(str(val_x) + val_y)
print(val_x + int(val_y))
print(val_x * 2 + val_y) # Careful with types here
```

10.

```
x = 17
y = 4
print(x / y)
print(x // y)
print(x % y) # Modulo
print(-17 // 4)
```

11.

```
a = 10
b = a
a = 20
print(a)
print(b)
```

12.

```
s = "IIT"
print(s * 3)
print(s + " " + "DELHI")
# print(s + 2025) <-- Would this line run? Why or why not?
```

13.

```
result = 10 + 3 * 2 ** 3 // 4 - 1
print(result)
```

Fill in the Code

Complete each program so that it performs the required task.

1. Read an integer from the user and print its square.

```
# read a number from user
num = int( _____ )
# print its square
print( _____ )
```

2. Read a name and age, then print: Name: <name>, Age: <age>

```
name = input("Enter your name: ")
age = input("Enter your age: ")
print("Name:", _____, "\n Age:", _____)
```

3. Read two float numbers and print their average.

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
avg = ( _____ ) / _____
print("Average is", avg)
```

4. Read a string and print it 3 times in a single line separated by spaces using *.

```
word = input("Enter a word: ")
result = (word + ___) * _____
print(result)
```

5. Read the length and breadth of a rectangle and print its area.

```
length = float(input("Enter length: "))
breadth = float(input("Enter breadth: "))
area = _____ * _____
print("Area:", area)
```

6. Read a number and print True if it is even, else print False using only arithmetic and comparison, no if.

```
num = int(input("Enter a number: "))
print( _____ == 0)
```

7. Read a 4-digit year and print the last two digits.

```
year = int(input("Enter year: "))
last_two = _____ % _____
print(last_two)
```

8. Hypotenuse Calculator (Pythagorean Theorem).

```
leg_a = float(input("Enter leg a: "))
leg_b = float(input("Enter leg b: "))
hypotenuse = ( _____ + _____ ) ** 0.5
print("The hypotenuse is:", hypotenuse)
```

9. Swap the values of x and y without using a temporary variable.

```
x = 10
y = 50
x, y = _____, _____
print("x:", x, "y:", y) # Output should be x: 50 y: 10
```

Write the Code

1. Write a program that takes your name as input and prints
Hello, <name>! where <name> is what the user typed.
2. Write a program that:
 - Asks the user for two integers.
 - Prints their sum and product on separate lines.
3. Write a program that:
 - Asks the user for a word and a number n in the range $[1, 5]$.
 - Prints the word repeated n times on one line using the $*$ operator.
4. Write a program that:
 - Asks the user for a total number of seconds (integer).
 - Convert this into hours, minutes, and remaining seconds.
 - Example Input: 3665.
 - Expected Output: 1 hours, 1 minutes, 5 seconds.

Chapter 2

Conditionals: If-Else

Output Prediction

1.

```
x = 7
if x % 2 == 0:
    print("Even")
else:
    print("Odd")
```

2.

```
marks = 85
if marks >= 90:
    print("A")
elif marks >= 80:
    print("B")
else:
    print("C")
```

3.

```
a = 5
b = 5
if a > b:
    print("a")
elif a < b:
    print("b")
else:
    print("equal")
```

4.

```
n = -3
if n > 0:
    if n % 2 == 0:
        print("Positive even")
    else:
        print("Positive odd")
else:
    print("Not positive")
```

5.

```
num = 15
if num % 3 == 0 and num % 5 == 0:
    print("FizzBuzz")
elif num % 3 == 0:
    print("Fizz")
elif num % 5 == 0:
    print("Buzz")
```

6.

```
x = 0
if x:
    print("True block")
else:
    print("False block")
```

Fill-ups / Equivalence / Correction

1. Fill the blanks to print the minimum of two numbers using if-else.

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))
if a < b:
    print("Minimum is", _____)
else:
    print("Minimum is", _____)
```

2. Check if the following two code fragments are equivalent for all integer x. If not, give an example value of x where they differ.

Fragment A:

```
if x > 10:
    print("Large")
else:
    print("Not large")
```

Fragment B:

```
if x >= 10:
    print("Large")
else:
    print("Not large")
```

3. Check if the following two code fragments are equivalent. If they are not, explain the difference.

Fragment A:


```

if n % 2 == 0:
    print("Even")
else:
    print("Odd")

```

Fragment B:

```

if n % 2 != 0:
    print("Odd")
else:
    print("Even")

```

4. Fill the missing condition so that the program prints Inside range only when n is between 1 and 100 (inclusive).

```

n = int(input("Enter n: "))
if _____:
    print("Inside range")
else:
    print("Outside range")

```

5. Complete the program to print Leap year if year is divisible by 400, or divisible by 4 but not by 100. Otherwise print Not leap year.

```

year = int(input("Enter year: "))
if ( _____ ) or ( _____ and _____ ):
    print("Leap year")
else:
    print("Not leap year")

```

6. Fill the blanks to classify a number as Even Positive, Odd Positive, or Non-positive.

```

n = int(input("Enter n: "))
if n > 0 and n % 2 == 0:
    print("Even Positive")
elif n > 0 and _____:
    print("Odd Positive")
else:
    print("Non-positive")

```

Write the Code

- Write a program that reads an integer and prints whether it is Positive, Negative, or Zero using if-elif-else.
- Write a program that reads a person's age and:
 - Prints Child if age < 13.

- Prints Teen if $13 \leq \text{age} \leq 19$.
 - Prints Adult otherwise.
3. Write a program that reads three integers and prints the largest number using nested if-else.
 4. Write a program to read a 3-digit number and count how many of its digits are odd, then print that count.

Chapter 3

While Loops

Output Prediction

1.

```
i = 1
while i <= 3:
    print(i)
    i = i + 1
```

2.

```
i = 1.0
while i <= 13:
    print(i * 2)
    i = i + 2.5
```

3.

```
i = 5
while i > 0:
    print(i)
    i = i - 2
```

4.

```
sum = 0
while i < 9:
    sum = sum + i
    i = i+1
print(sum)
```

5.

```
i = 1
while i < 5:
    print("Loop", i)
    i = i + 1
print("End")
```

Fill / Errors / Infinite Loops

1. The following program has three errors. Rewrite it correctly so that it prints numbers 10, 9, 8, 7, 6.

```
i = 10
while i > 5:
    print(i)
    i = i - 1
```

2. Fill in the blanks to compute the factorial of a positive integer n using a while loop.

```
n = int(input("Enter n: "))
fact = 1
i = 1
while i <= n:
    fact = fact * _____
    i = _____
print("Factorial is", fact)
```

3. Predict what the following infinite loop will print in the first five lines.

```
i = 1
while True:
    print(i)
    i = i + 1
    # assume user never stops the program
```

4. Fill the blanks to count the number of digits in a positive integer n .

```
n = int(input("Enter n: "))
count = 0
while n > 0:
    count = count + 1
    n = _____
print("Number of digits:", count)
```

5. Fill in the blanks so that the program reads integers from the user one by one and stops when the sum reaches or exceeds 100. Print the final sum.

```
total = 0
while _____ < 100:
    num = int(input("Enter a number: "))
    total = total + _____
print("Final sum:", total)
```

6. Fill in the blanks so that the program reads an integer n and prints a pattern of stars. Each line i (from 1 to n) has i stars.

```
n = int(input("Enter n: "))
i = 1
while i <= _____:
    print("*" * _____)
    i = _____
```

7. Fill in the blanks so that the program computes a^b (a raised to the power b) using a while loop.

```
a = int(input("Enter base: "))
b = int(input("Enter exponent: "))
result = 1
i = 0
while i < ____:
    result = result * ____
    i = ____
print(f"{a} raised to {b} is {result}")
```

8. Fill in the blanks so that the program reads an integer and reverses its digits using a while loop (e.g., input 1234 → output 4321).

```
n = int(____)
rev = 0
while n > 0:
    digit = ____
    rev = rev * 10 + digit
    n = ____
print(rev)
```

Programming Exercises

- Write a program using a while loop to print the squares of numbers from 3 to 8.
- Write a program that reads a positive integer from the user and calculates the sum of all its digits using a while loop. For example, if the input is 1234, the output should be $1 + 2 + 3 + 4 = 10$.
- Write a program that reads a positive integer and counts how many times digit 1 appears in that number using a while loop. For example:
 - Input: number = 112211, digit to count = 1
 - Output: Digit 1 appears 4 times
- Write a program that reads an integer and checks if it is a palindrome (reads the same forwards and backwards) using a while loop. Print Palindrome or Not a palindrome. For example, 121 is a palindrome, but 123 is not.

Chapter 4

For Loops, Break Statements, and Nested Loops

Note: The end parameter inside a print statement is used to specify a string to be printed at the end, enabling multiple outputs to appear on the same line or with custom separators. For eg.

```
print("Hello", end = " ")  
print("World")
```

prints

Hello World

If we do not specify the end parameter, the newline character is printed at the end.

```
print("Hello")  
print("World")
```

prints

Hello
World

Output Prediction

1.

```
for i in range(1, 5):  
    for j in range(1, 3):  
        print(i, j)
```

2.

```
for i in range(3):  
    for j in range(i):  
        print(j, end=" ")  
    print()
```

3.

```
for i in range(1, 6):
    if i == 3:
        break
    for j in range(1, 4):
        print(i * j)
```

4.

```
for i in range(1, 4):
    for j in range(1, 4):
        if i + j == 5:
            break
        print(i, j)
```

5.

```
for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            break
        print(i, j)
```

6. Predict the output.

```
for i in range(1, 4):
    for j in range(1, 4):
        if i == j:
            break
        print(i, j)
```

7. Predict the output of the following code.

```
for i in range(1, 5):
    for j in range(1, 5):
        if j > i:
            break
        print(j, end=" ")
    print()
```

Fill / Errors / Break Logic

1. The following program has errors. Rewrite it so that it prints a 3 X 3 grid of stars.

```
for i in range(3)
    for j in range(3):
        print("*")
```

2. Fill in the blanks to print the following pattern:

```
1
12
123
1234
```

```
for i in range(1, 5):
    for j in range(1, _____):
        print(j, end=" ")
    print()
```

3. Fill in the blanks so that the program finds the first pair (i, j) such that $i * j = 12$ and stops both loops.

```
for i in range(1, 7):
    for j in range(1, 7):
        if i * j == 12:
            print(i, j)
            _____
    _____
```

4. Fill in the blanks so that the loop prints numbers from 1 to 20 but stops completely when a multiple of both 4 and 6 is found.

```
for i in range(1, 21):
    if i % 4 == 0 and i % 6 == 0:
        print(i)
        _____
    print(i)
```

5. The following program is intended to stop printing as soon as a pair (i, j) satisfies $i + j = 10$. However, it contains errors. Identify and fix the errors.

```
for i in range(1, 6):
    for j in range(1, 4):
        if i + j = 10:
            break
        print(i, j)
```

Write the Code

- Write a program using nested for loops to print a multiplication table from 1 to 5.
- Write a program to print the following pattern using nested loops:

```
*
**
```


3. Write a program that reads two integers r , s finds the first prime number between r and s , and stops immediately after finding it.
4. Write a program that checks whether a given number k is perfect (sum of proper divisors of it equals the number) using a for loop.
5. Write a program that prints all prime factors of a number n sorted in increasing order using nested for loops and stops early if the count of prime factors exceeds five.
6. Write a program that reads a number n and uses for loop to separate its digits into two new numbers:
 - One formed by all odd digits (in the same order as in the original number).
 - One formed by all even digits.
 - Print both numbers. (If there are no digits of a type, print 0 for that number.)

For example, if the input is "1103456789", then the output must be "113579 468".

Chapter 5

Strings and String Slicing

Output Prediction

1.

```
s = "computer"
print(s[1:4])
print(s[:3])
print(s[5:])
```

2.

```
s = "programming"
print(s[-1])
print(s[-5:-2])
```

3.

```
s = "abcdef"
print(s[:2])
print(s[1:2])
```

4.

```
s = "python"
print(s[: -1])
print(s[1:5:2])
```

5.

```
s = "iteration"
for ch in s:
    print(ch, end="-")
```

6. Predict the output.

```
s = "science"
for i in range(len(s)):
    if i % 2 == 0:
        print(s[i], end=" ")
```

7. Predict the output of the following code.

```
s = "datastructures"
print(s[3:10])
print(s[10:3:-1])
```

Fill / Errors / String Logic

1. The following program has errors. Rewrite it so that it prints each character of the string on a new line.

```
s = "python"
for i in range(len(s))
print(s[i], end = "")
```

2. Fill in the blanks to print the first half of the string s.

```
s = input()
print(s[_____])
```

3. Fill in the blanks so that the program checks whether a string is a palindrome.

```
s = input()
if s == _____:
    print("Palindrome")
else:
    print("Not Palindrome")
```

4. Fill in the blanks so that the loop counts the number of vowels in a string.

```
s = input()
count = 0
for ch in s:
    if ch in _____:
        count += 1
print(count)
```

5. The following program is intended to print all substrings of length 3, but it contains errors. Identify and fix the errors.

```
s = "abcdef"
for i in range(0, len(s)):
print(s[i:i+3])
```

Write the Code

1. Write a program that reads a string and prints its length without using the `len()` function.
2. Write a program to count the number of uppercase letters, lowercase letters, digits, and spaces in a given string.

For example, for input "code", the output should be:

```
c
co
cod
code
```

3. Write a program that reads a string and creates a new string by swapping the case of each character (uppercase to lowercase and vice versa).
4. Write a program that reads a string `s` and an integer `k`, and prints every `k`-th character of the string using slicing.

Chapter 6

Functions and Basic Recursion

Output Prediction

1.

```
def f(x):  
    x = x + 1  
    return x  
  
a = 5  
print(f(a))  
print(a)
```

2.

```
def mystery(n):  
    if n == 0:  
        return 0  
    return n + mystery(n-1)  
  
print(mystery(4))
```

3.

```
def show(n):  
    if n == 0:  
        return  
    print(n)  
    show(n-1)  
show(3)
```

4. Predict the output of the following code.

```
def rec(n):  
    if n <= 3:  
        return 1  
    return n * rec(n-1)  
  
print(rec(7))
```

5. Predict the output.

```
def fun(a, b):
    return a * b
print(fun(2, 3 + 1))
```

Fill / Errors / Function Logic

1. The following program produces an error, Explain why.

```
x = 10
def update():
    x = x + 5
update()
print(x)
```

2. Fill in the blanks to write a recursive function that prints numbers from n to 1 in different lines.

```
def print_down(n):
    if -----:
        return
    -----
    -----
```

3. The following recursive program is intended to compute the sum of digits of a number, but it contains multiple errors (logical and syntactic). Identify and fix them.

```
def sum_digits(n):
    if n == 0:
    return 0
    return n % 10 + sum_digits(n / 10)
```

4. Fill in the blanks so that the program correctly returns the number of times the helper function is executed. Do not use any global variables.

```
def counter():
    count = 0
    def helper():
        new_count = ----- + 1
        return new_count
    return helper()

print(counter())
```

5. The following program intends to compute the sum of numbers from 1 to n using functions, but it contains a logical error related to variable scope. Fill in the blanks to fix it without using global variables.

```
def outer(n):
    total = 0
    def add(i):
        return total + i
    for i in range(1, n + 1):
        total = -----
    return total

print(outer(4))
```

Write the Code

1. Write a function that checks whether a given string is a palindrome using recursion.
2. Write a function that takes a positive integer n as input and returns a string which is the binary representation of n .
3. Write a function that takes two integers a and b and returns their greatest common divisor (GCD) using recursion.
4. Write a recursive function that computes the number of trailing zeros in $n!$ (factorial of n). Do not compute the factorial explicitly.
5. Imagine a robot sitting at the top-left corner $(0,0)$ of an $m \times n$ grid. The robot can only move in two directions: Right or Down. Write a recursive function that takes non-negative integers m, n as input and outputs the total number of unique paths the robot can take to reach the bottom-right corner (m, n) . *Bonus: Can you derive a closed-form expression for the number of unique paths in terms of m, n ?*
6. Imagine n people standing in a circle, numbered 1 to n . Starting from person 1, you move around the circle and eliminate every k -th person. Once a person is eliminated, the circle closes up, and the count continues from the next available person.
 eg. When $n = 7, k = 3$: First the 3rd person is eliminated, followed by the 6th person, followed by 2nd, 7th, 5th, 1st persons. The 4th person is the survivor.
 Write a function that takes positive integers n, k as input ($k \leq n$) and outputs the position of the survivor.
Easier problem: First solve it when $k = 2$.
7. Write a function, which when given a string s , prints all possible arrangements of s each in a new line (eg. on input "ABC", it prints "ABC", "ACB", "BAC", "BCA", "CAB", "CBA" each in a new line). You may assume that the string consists of distinct characters. You may print the arrangements in any order. *Bonus: Now do not assume the characters are distinct, and print the arrangements in lexicographic (dictionary) order.*

Chapter 7

Binary Search on Integers and Strings

Output Prediction

1.

```
def g(n):
    l, r = 1, n
    ans = 0
    while l <= r:
        m = (l+r)//2
        if 3*m + 7 <= n:
            ans = m
            l = m + 1
        else:
            r = m - 1
    return ans

print(g(100))
```

2.

```
def f(n):
    l, r = 1, n
    while l <= r:
        m = (l+r)//2
        if m*m*m <= n:
            l = m + 1
        else:
            r = m - 1
    return r

print(f(100))
```

3.

```
def g(a, b):
    steps = 0
    while b:
        a, b = b, a % b
        steps += 1
    return steps
```



```
print(g(144, 89))
```

Fill / Errors / Function Logic

1. The following program may enter an infinite loop. Explain precisely why.

```
def bad1(n):
    l, r = 0, n
    while l < r:
        m = (l+r)//2
        if m*m <= n:
            l = m
        else:
            r = m - 1
    return l
```

2. Find the smallest non-negative integer x such that

$$x^3 + 5x^2 + 2 \geq \frac{n}{x+1}$$

Assume n is a positive integer, $n \leq 10^6$.

Fill in the blanks.

```
def tricky(n):
    l, r = 0, 10**6
    ans = -1

    while l <= r:
        m = (l+r)//2

        left = _____
        right = _____

        if _____:
            ans = m
            _____
        else:
            _____

    return ans
```

3. The following function attempts to find the smallest integer x such that $x^3 \geq n$, but may skip the correct answer or never find it. Why?

```
def bad2(n):
    l, r = 0, n
```

```

ans = -1
while l < r:
    m = (l+r)//2
    if m*m*m >= n:
        ans = m
        r = m - 1
    else:
        l = m
return ans

```

4. Consider the function:

$$f(x) = |x^2 - 10x + 21|$$

We attempt to search for the smallest non-negative integer x such that $f(x) \leq 3$. Explain why the binary search technique will not work here.

[Hint: Look at the graph of $f(x)$.]

Write the Code

1. Implement the square root finding binary search function using recursion.
2. A baker has W grams of flour. To make a single giant cookie of radius r , they need r^2 grams of flour. What is the largest integer radius cookie they can make if they want to make C identical cookies? Write a function that takes as input W , C and outputs the largest integer radius r .
3. Two traffic signals blink every a seconds and b seconds respectively. At time 0, both blink simultaneously. Find the number of seconds t until they blink together again.

Write a function that takes as input integers a and b , and returns t .

Then, given a time T , determine how many times m they blink together within the first T seconds (including time 0).

Write a function that takes as input integers a , b , T , and returns m .

4. Write a function that takes integers P, Q as input, and outputs the number of points with integer coordinates on a line segment connecting $(0,0)$ and (P,Q) .
5. You are given integers a and b .

Find the smallest positive integer x such that:

$$\gcd(a+x, b+x) > 1.$$

If no such x exists, print -1 .

-
6. Write a function that, on input integer K , returns the largest integer x such that $x^5 + x \leq K$.
 7. A plane needs to travel D kilometers. It has two engines. Engine A consumes x^3 liters per km, and Engine B consumes x^2 liters per km, where x is the speed in km/min. Given a total fuel limit of F liters, find the maximum constant speed x the plane can maintain to reach its destination.
Assume D, F, x are restricted to be positive integers. Write a function that takes D, F as input and finds x .
 8. Given a positive integer n , find two positive integers a and b such that $a + b = n$ and $\text{lcm}(a, b)$ is minimized. ($\text{lcm}(a, b)$ is the least common multiple of a and b .)
Bonus: Can you solve this using a loop that runs at most $\sqrt{n}$ times?

Chapter 8

Lists and Tuples

Output Prediction

1.

```
def outer(L):  
    def inner(k):  
        L.extend([k, -k])  
    for i in range(2):  
        inner(i)  
    return L  
  
arr = [10]  
res = outer(arr)  
print(res)
```

2.

```
lst = [2, 5, 1, 7, 6, 3, 4]  
a = lst.pop()  
print(a)  
b = lst.pop(a)  
print(b)  
lst.sort()  
lst.append(a)  
lst.insert(a, b)  
print(lst)
```

3.

```
def f(a):  
    a.append(4)  
    a = [9, 88, 0, 9]  
    a.remove(0)  
    a.append(10)  
  
x = [1, 2, 3]  
f(x)  
print(x)
```

4.

```
t = (1, 2, 3)
print(t[1])
t.pop()
print(t, len(t))
```

5.

```
def outer():
    lst = [1, 2]
    def inner():
        lst.append(3)  # Modifies outer list
        return lst
    return inner()

result = outer()
print(result)
```

6.

```
x = [1, [2, 3], 4]
y = x[:]
y[1][0] = 99
print(x)
print(y)
```

7.

```
a = [[0], [1], [2]]
b = a * 2
c = a[:]
b[1].append(99)
c[2] = ['x']
print(a)
print(b)
print(c)
```

8.

```
A = [[]]
C = A + A
C[0].append(0)
C[1].append(1)
print("A = ", A)
print("C = ", C)
```

9.

```
def square_plus_one(x):
    print(id(a)==id(x))
    print(id(a[0])==id(x[0]))
    y = (x[0] * x[0] + 1,)
    return y

a = (1001,)
b = square_plus_one(a)
print(b[0])
```

Complete the Code

1. Complete recursive tuple reversal:

```
def reverse_tuple(t):
    if not t:
        return ()
    return reverse_tuple(____) + (t[0],)

t=(1,3,5)
print(reverse_tuple(t))
```

2. Suppose we want to visit all elements of a **matrix in spiral order**. For example, if the matrix is: `[ [1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12] ]`
Then the spiral traversal is: `1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7`
Fill in the blanks.

```
def spiral(mat):
    result = []
    top = 0
    bottom = len(mat) - 1
    left = 0
    right = len(mat[0]) - 1

    while top <= bottom and left <= right:

        # Traverse left -> right
        for i in range(_____, _____ + 1):
            result.append(mat[top][i])
        top += 1

        # Traverse top -> bottom
        for i in range(_____, _____ + 1):
            result.append(mat[i][right])
        right -= 1

        # Traverse right -> left
        if top <= bottom:
            for i in range(_____, _____ - 1, -1):
                result.append(mat[bottom][i])
            bottom -= 1

        # Traverse bottom -> top
        if left <= right:
            for i in range(_____, _____ - 1, -1):
                result.append(mat[i][left])
            left += 1

    return result
```

3. Complete the code below to compute the trace of a product of two matrices.

```
def efficient_trace_of_product(A, B):
    # Assume (AB) is a square product
    n = len(A)
    m = len(_____)
    trace = 0

    for i in range(n):
        for j in range(_____):
            trace = _____ + _____ * _____

    return trace

# Example
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]
print(efficient_trace_of_product(A, B)) # Result: 70
```

4. Complete the code below to calculate the average score for each product in List A by manually scanning and matching entries from the reviews in List B.

```
def print_product_averages(list_a, list_b):
    for target_product in list_a:
        total_score = _____
        count = 0

        for name, product_name, rating in list_b:
            if product_name == _____:
                total_score = total_score + _____
                count = _____

        if _____ > 0:
            average = _____ / _____
            print(target_product, "rating:", _____)
        else:
            print("No reviews found")

A = ["Nokia 105 phone", "Asus Vivobook", "Casio watch"]
B = [
    ("Alice", "Nokia 105 phone", 5),
    ("Bob", "Nokia 105 phone", 2),
    ("Charlie", "Asus Vivobook", 4),
    ("David", "Nokia 105 phone", 5)
]

print_product_averages(A, B)
```

Write the Code

1. Write a function `mult(A, B)` that takes as input two matrices (expressed as list of lists) and outputs $C = A \times B$. If the dimensions of A and B are not compatible for matrix multiplication, then it outputs -1 .
2. Write a function `prime_list` that takes as input a positive integer n , and outputs a list L that contains all primes less than n .
3. Write a function `tupleCheck(L, k)` that takes as input a tuple T , a positive integer k , and outputs `True` if there exists an index i such that

$$T[i] \leq T[i+1] \leq \dots \leq T[i+k-1]$$

(that is, the elements in the sub-tuple $T[i : i+k]$ are non-decreasing).

For example, if $T = [44, 33, 22, 44, 55, 11]$, then `tupleCheck(T, 3)` should return `True`, but `tupleCheck(T, 4)` should return `False`.

4. Write a function `SetDiff(L, M)` that takes as input two lists L, M containing distinct elements and outputs all elements that are present in L but not present in M .

Chapter 9

Recursions using Lists and Tuples

Recursion Tree

Draw recursion tree for the following codes.

1.

```
def sum_list(lst, acc=0):
    if not lst:
        return acc
    return sum_list(lst[1:], acc + lst[0])

print(sum_list([1,2,3,4])) # 10
```

2.

```
def filter_even(lst, acc=None):
    if acc is None:
        acc = []
    if not lst:
        return acc
    if lst[0] % 2 == 0:
        acc = acc + [lst[0]]
    return filter_even(lst[1:], acc)

print(filter_even([1,2,3,4,5,6]))
```

3.

```
def flatten(lst, acc=None):
    if acc is None: acc = []
    if not lst: return acc
    first = lst[0]
    rest = lst[1:]

    if isinstance(first, list):
        return flatten(first + rest, acc)
    else:
        return flatten(rest, acc + [first])

print(flatten([1,[2,3],[4,[5]]]))
```

4.

```
def strings(chars, length, current):
    if current is None:
        current = []

    if length == 0:
        print(current)
        return

    for c in chars:
        strings(chars, length - 1, current + [c])

strings(['a', 'b'], 3, None)
```

5.

```
def no_consecutive_ones(n, current=None):
    if current is None: current = []

    if n == 0:
        print(current)
        return

    no_consecutive_ones(n - 1, current + [0])

    if not current or current[-1] == 0:
        no_consecutive_ones(n - 1, current + [1])

no_consecutive_ones(3)
```

6.

```
def compute(n, ones_needed, current=None):
    if current is None: current = []

    if n == 0:
        if ones_needed == 0:
            print(current)
            return

    if ones_needed > n:
        return

    compute(n - 1, ones_needed, current + [0])

    if ones_needed > 0:
        compute(n - 1, ones_needed - 1, current + [1])

compute(4, 2)
```

Complete the Code

1. Complete the code below to print all ordered partitions of 6 that consists of either all odd elements or all even elements.

```
def compute(prefix, m):
    if (m == 0):
        print(prefix)
        return

    for i in range(1, m+1):
        if len(_____)==0 or _____ %2 == 0:
            compute(prefix+_____, _____)

compute([], 6)
```

2. Complete the code below to print a list whose elements are all permutations of string 'abc'.

```
def permute(s):
    if len(s) <= 1:
        return [s]

    perms = []
    for i in range(len(s)):
        ch = _____
        rest = s[:i] + s[i+1:]
        for p in permute(_____):
            perms.append(_____)

    return perms

S = "abc"
print(permute(S))

# output: ["abc", "acb", "bac", "bca", "cab", "cba"]
```

3. Complete the code below to print all ordered partitions of n of length k .

Note that for $n=5$ and $k=3$, the output must be:

```
[1, 1, 3]
[1, 2, 2]
[1, 3, 1]
[2, 1, 2]
[2, 2, 1]
[3, 1, 1]
```

```
def compute(n, k, current):

    if k == 1:
        if n > 0:
            print(current + _____)
        return

    for i in range(1, n - k + 2):
        compute(_____, _____, current + _____)

n=5
k=3
compute(n, k, [])
```

4. An unordered partition of integer n into k parts is a way of writing n as a sum of k positive integers where order does not matter: $n = a_1 + a_2 + \dots + a_k$ with $a_1 \geq a_2 \geq \dots \geq a_k \geq 1$.

Complete the recursive function below to print all unordered partitions of n of length k :

Note that for $n=9$ and $k=3$, the output must be:

```
[7, 1, 1]
[6, 2, 1]
[5, 3, 1]
[5, 2, 2]
[4, 4, 1]
```

```
[4, 3, 2]
[3, 3, 3]
```

```
def unordered_partitions(n, k, max_val, current):
    if k == 1:
        if 1 <= n <= max_val:
            print(current + [n])
        return

    for i in range(min(n-(k-1), max_val), 0, -1):
        unordered_partitions(____, ____, ____, current+____)

n=9
k=3
unordered_partitions(n, k, n, [])
```

5. Complete the recursive function below to print all permutations of a list *L*:

```
def permute(i, k, S):
    if ____:
        print(S)
    else:
        for j in range(i, len(L)):
            L[i], L[j] = L[j], L[i]
            permute(____, _____, S+[____,])
            L[i], L[j] = L[j], L[i]

L = [1, 2, 3]
permute(0, len(L), [])
```

For example, for *L* = [1, 2, 3], the output must be:

```
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
```

[3, 2, 1]

[3, 1, 2]

Chapter 10

Aliasing and References

Output Prediction

1.

```
a = [1, 2, 3]
b = a
b.append(4)
b = b + [5]
print(a)
print(b)
```

2.

```
def curve(scores):
    scores = scores + [10]
    for i in range(len(scores)):
        scores[i] += 5
    return scores

original_scores = [70, 80, 90]
curved = curve(original_scores)
print(original_scores)
print(curved)
```

3.

```
ranking = ["Alice", "Bob", "Charlie"]
board_a = ranking
board_b = ranking[:]

board_a.insert(0, "Zara")
board_b.append("David")
ranking[1] = "Eve"

print(ranking)
print(board_a)
print(board_b)
```

4.

```
def f(x, data=[]):
```

```

        data.append(x)
        return data

print(f(1))
print(f(2))
print(f(3, []))
print(f(4))

```

5.

```

original = [[1, 2], [3, 4], [5, 6]]
shallow = original[:]
deep = [row[:] for row in original]

original[0].append(99)
original.append([7, 8])

print(original)
print(shallow)
print(deep)

```

6.

```

gradebook = [[85, 90], [70, 75], [95, 88]]
backup = gradebook[:]

gradebook[0].append(100)
gradebook[1] = [0, 0]

print(gradebook)
print(backup)

```

7.

```

def swap(lst, i, j):
    lst[i], lst[j] = lst[j], lst[i]

def no_swap(lst, i, j):
    lst = lst[:]
    lst[i], lst[j] = lst[j], lst[i]
    return lst

data = [10, 20, 30, 40]
swap(data, 0, 3)
result = no_swap(data, 1, 2)
print(data)
print(result)

```

8.

```

def enqueue(queue, item):
    queue.append(item)

def snapshot(queue):
    return queue[:]

```



```

q1 = [1, 2, 3]
q2 = snapshot(q1)
enqueue(q1, 4)
q2.append(99)
q1[0] = 0

print(q1)
print(q2)

```

9.

```

def mystery(lst):
    for i in range(len(lst)):
        lst[i] = lst[i] * 2
    lst = [0] * len(lst)

nums = [1, 2, 3, 4]
mystery(nums)
print(nums)

```

Complete the Code

1. Complete the function so that it returns a **new** list with each element doubled, without modifying the original list.

```

def double_copy(lst):
    result = -----
    for i in range(len(lst)):
        result[i] = -----
    return result

original = [3, 1, 4, 1, 5]
doubled = double_copy(original)
print(original)  # Should be unchanged: [3, 1, 4, 1, 5]
print(doubled)   # [6, 2, 8, 2, 10]

```

2. Complete the function to perform a **deep copy** of a 2D list (list of lists) *without* using the copy module.

```

def deep_copy_2d(matrix):
    result = []
    for row in matrix:
        new_row = -----
        result.-----
    return result

A = [[1, 2, 3], [4, 5, 6]]
B = deep_copy_2d(A)
B[0][0] = 99

```

```
print(A[0][0])  # Should still be 1
print(B[0][0])  # Should be 99
```

3. A student wants to collect multiple filtered versions of a dataset without the versions interfering with each other. Complete the function so each filtered result is stored as an **independent** copy.

```
def collect_filters(data, thresholds):
    results = []
    for t in thresholds:
        filtered = []
        for x in data:
            if x >= t:
                filtered.append(x)
        results.append(filtered)
    return results

data = [3, 7, 1, 9, 4, 6]
snapshots = collect_filters(data, [4, 6, 8])
data.append(100)
print(snapshots)
# [[7, 9, 4, 6], [7, 9, 6], [9]] -- data change must NOT
  ↪ affect results
```

4. The function below is supposed to remove all duplicates from a list **in-place** (modifying the original list), preserving the first occurrence of each element. Fill in the blanks.

```
def remove_duplicates_inplace(lst):
    seen = {}
    i = 0
    while i < len(lst):
        if lst[i] in seen:
            lst.remove(lst[i])
        else:
            seen[lst[i]] = True
            i += 1

data = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
remove_duplicates_inplace(data)
print(data)  # [3, 1, 4, 5, 9, 2, 6]
```

Write the Code

1. A bank maintains a list of account balances. Write a function `apply_transaction(balances, index, amount)` that returns a **new** list reflecting the transaction (adding amount to `balances[index]`), without modifying the original `balances` list. Then write a

second function `transaction_log(balances, transactions)` that applies a list of `(index, amount)` pairs one by one, returning a list of snapshots — one per transaction — where each snapshot is independent.

2. Write a function `track_history(steps)` that takes a list of lists `steps`, where each element represents the state of some process at one point in time. The function should return a new list of lists where each inner list is an **independent copy** of the corresponding step, so that modifying the originals later does not corrupt the history. Demonstrate that your function works correctly by modifying steps after calling it and showing the history is unaffected.
3. Write a function `flatten_copy(nested)` that takes a list of lists and returns a single flat list containing all the elements. The function must not modify any of the inner lists or the outer list.
4. Write a function `rotate_inplace(lst, k)` that rotates the list `lst` to the right by `k` positions **in-place** (i.e., modifying the original list and returning `None`). For example, rotating `[1, 2, 3, 4, 5]` by 2 gives `[4, 5, 1, 2, 3]`.

Chapter 11

Invariants

Output Prediction

For each code snippet, determine what is printed. Before tracing, identify the loop invariant that is maintained at each iteration.

1.

```
def fun1(lst):
    best = lst[0]
    result = []
    for x in lst:
        if x > best:
            best = x
        result.append(best)
    return result

print(fun1([3, 1, 4, 1, 5, 9, 2, 6]))
```

2.

```
def mystery(n):
    lo, hi = 0, n
    while lo < hi:
        mid = (lo + hi) // 2
        if mid * mid <= n:
            lo = mid + 1
        else:
            hi = mid
    return lo - 1

print(mystery(25))
print(mystery(26))
```

3.

```
def process(lst):
    i, j = 0, len(lst) - 1
    while i < j:
        if lst[i] + lst[j] == 10:
            print(lst[i], lst[j])
```

```

        i += 1
        j -= 1
    elif lst[i] + lst[j] < 10:
        i += 1
    else:
        j -= 1

process([1, 3, 4, 5, 6, 7, 9])

```

4.

```

def transform(lst):
    n = len(lst)
    for i in range(n):
        for j in range(0, n - i - 1):
            if lst[j] > lst[j + 1]:
                lst[j], lst[j + 1] = lst[j + 1], lst[j]
    return lst

print(transform([5, 3, 8, 1, 2]))

```

5.

```

def compress(lst):
    if not lst:
        return []
    result = [lst[0]]
    for i in range(1, len(lst)):
        if lst[i] != result[-1]:
            result.append(lst[i])
    return result

print(compress([1, 1, 2, 3, 3, 3, 2, 2, 1]))

```

6.

```

def encode(s):
    if not s:
        return []
    result = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            count += 1
        else:
            result.append((s[i - 1], count))
            count = 1
    result.append((s[-1], count))
    return result

print(encode("aaabbbccdddee"))

```

Complete the Code

1. The function below performs a sort. The invariant is: *after each outer iteration i , $lst[0:i+1]$ is sorted*. Fill in the inner loop so the invariant holds.

```
def sort(lst):
    for i in range(1, len(lst)):
        key = lst[i]
        j = -----
        while j >= 0 and lst[j] ----- key:
            lst[j + 1] = -----
            j -= 1
        lst[-----] = key
    return lst

print(sort([5, 2, 4, 6, 1, 3])) # [1, 2, 3, 4, 5, 6]
```

2. A two-pointer function finds all pairs in a **sorted** list that sum to a given target. The invariant is: *any valid pair that has been missed so far cannot have both elements between $lst[lo]$ and $lst[hi]$* . Fill in the blanks.

```
def find_pairs(lst, target):
    lo, hi = -----, -----
    pairs = []
    while lo < hi:
        s = lst[lo] + lst[hi]
        if s == target:
            pairs.append((lst[lo], lst[hi]))
            lo += 1
            -----
        elif s < target:
            -----
        else:
            -----
    return pairs

print(find_pairs([1, 2, 3, 4, 5, 6, 7, 8, 9], 10))
# [(1, 9), (2, 8), (3, 7), (4, 6)]
```

3. The function below is supposed to remove all elements greater than k from a list **in-place**, maintaining the invariant: *$lst[0:i]$ contains only elements $\leq k$ at all times*. Fill in the blanks.

```
def remove_large(lst, k):
    i = -----
    while i < len(lst):
        if lst[i] ----- k:
            lst.pop(-----)
        else:
            -----
```

```

    return lst

print(remove_large([3, 7, 1, 8, 2, 5, 4], 4)) # [3, 1, 2, 4]

```

Write the Code

1. **Balanced Brackets.** Write a function `is_balanced(s)` that takes a string of brackets `((, ), [, ], {, })` and returns `True` if they are properly balanced and nested. Maintain the invariant: *at every point during the scan, the stack contains exactly the unmatched opening brackets seen so far, in order.*

2. **Tile Flipping.** You have a row of n tiles, each either White (W) or Black (B). In one move, you may flip any tile (W becomes B, and vice versa), along with both of its immediate neighbours (if they exist).

Show that the **parity of the number of Black tiles** is an invariant of each move. Use this invariant to determine: starting from the configuration `WWWWW` (5 white tiles), is it possible to reach the configuration `BBBBB` (5 black tiles)?

3. **Token Game.** A bag contains 10 red tokens and 7 blue tokens. At each step, you pick two tokens at random. If they are the same colour, you remove both and put one red token back. If they are different colours, you remove both and put one blue token back. Repeat until only one token remains.

Identify an invariant that lets you determine the colour of the last remaining token without simulating the game. What colour is it, and why?

4. **Sorted Merge Correctness.** You are given two sorted lists A and B and the following merge procedure:

```

def merge(A, B):
    result = []
    i, j = 0, 0
    while i < len(A) and j < len(B):
        if A[i] <= B[j]:
            result.append(A[i]); i += 1
        else:
            result.append(B[j]); j += 1
    result.extend(A[i:])
    result.extend(B[j:])
    return result

```

State a loop invariant for the `while` loop. Use it to argue that `result` is sorted and contains exactly all elements of A and B when the loop ends.

5. **Binary Search Correctness.** Consider the following binary search implementation:

```

def binary_search(lst, target):
    lo, hi = 0, len(lst) - 1

```

```
while lo <= hi:
    mid = (lo + hi) // 2
    if lst[mid] == target:
        return mid
    elif lst[mid] < target:
        lo = mid + 1
    else:
        hi = mid - 1
return -1
```

State the loop invariant precisely (in terms of where target can be in `lst`). Use it to argue that the algorithm always returns the correct index, or -1 if the target is absent.

Chapter 12

Dictionaries (Hash Maps)

Output Prediction

For each code snippet, determine what is printed.

1.

```
def freq(lst):
    d = {}
    for x in lst:
        if x in d:
            d[x] += 1
        else:
            d[x] = 1
    return d

print(freq([1,2,1,3,2,1]))
```

2.

```
def accumulate(lst):
    d = {}
    total = 0
    for i, x in enumerate(lst):
        total += x
        d[i] = total
    return d

print(accumulate([2, 4, 6]))
```

3.

```
def first_occurrence(s):
    d = {}
    for i, ch in enumerate(s):
        if ch not in d:
            d[ch] = i
    return d

print(first_occurrence("banana"))
```

4.

```
def pair_sum(lst, target):
    seen = {}
    for x in lst:
        if target - x in seen:
            print(x, target - x)
        seen[x] = True

pair_sum([1, 4, 5, 6, 3], 7)
```

5.

```
def group_by_parity(lst):
    d = {"even": [], "odd": []}
    for x in lst:
        if x % 2 == 0:
            d["even"].append(x)
        else:
            d["odd"].append(x)
    return d

print(group_by_parity([1,2,3,4,5]))
```

Complete the Code

1. **Word Frequency Counter.** The function should count how many times each word appears in the list and return a dictionary mapping words to their frequencies.

```
def word_count(words):
    d = {}
    for w in words:
        if _____ :
            d[w] += 1
        else:
            d[w] = _____
    return d

print(word_count(["a", "b", "a", "c", "b", "a"]))
```

2. **Two Sum (Return Indices).** The function should return the indices of two numbers that add up to the given target. You may assume exactly one valid pair exists.

```
def two_sum(lst, target):
    d = {}
    for i, x in _____:
        if target - x in d:
            return (_____, i)
        d[x] = _____
    return None
```

```
print(two_sum([2,7,11,15], 9))
```

3. **Character Transformation.** The function should replace characters in the string using the given mapping. If a character is not in the mapping, it should remain unchanged.

```
def transform(s, mapping):
    result = ""
    for ch in s:
        if ch in mapping:
            result += mapping[ch]
        else:
            result += ch
    return result

print(transform("abc", {"a": "x", "b": "y"}))
```

Word Problems

1. **Secret Code Reconstruction.** You are given a long string s and a shorter string p (the pattern). Find all starting indices in s where a substring of length $|p|$ is a rearrangement (permutation) of p .

In other words, identify all positions where the substring contains exactly the same characters as p , possibly in a different order.

Example: $p = \text{"abc"}$, $s = \text{"cbabadcbabbcbabaabccbabcb"}$

The output should be all indices i such that the substring $s[i : i + |p|]$ is a permutation of p .

2. **Subarray Sum Equals K.** Given a list of integers and a number k , count how many contiguous subarrays have sum equal to k .
3. **Longest Substring Without Repeating Characters.** Given a string, find the length of the longest substring with all distinct characters.
4. **Minimum Window Substring.** Given two strings s and t , find the smallest substring of s that contains all characters of t (including multiplicities).
5. **Smallest Subarray with All Distinct Elements.** Given an array, find the length of the smallest subarray that contains all distinct elements present in the entire array.
6. **Longest Substring with At Most K Distinct Characters.** Given a string and integer k , find the length of the longest substring that contains at most k distinct characters.

7. **Circular Route Check.** A delivery vehicle travels in a loop, with gains/losses of fuel at each stop.

Determine whether there exists a starting point such that the vehicle can complete the full loop without running out of fuel.

Chapter 13

Time Complexity

Complete the Code

1. The function below is meant to find the index of the **first** element in a sorted list that is greater than or equal to a given value (a *lower bound* search). Fill in the blanks so that it runs in $O(\log n)$ time.

```
def lower_bound(lst, val):
    lo, hi = 0, -----
    while lo ----- hi:
        mid = -----
        if lst[mid] < val:
            lo = -----
        else:
            hi = -----
    return lo

print(lower_bound([1, 3, 3, 5, 7, 9], 3)) # 1
print(lower_bound([1, 3, 3, 5, 7, 9], 4)) # 3
```

How many primitive comparisons does this perform in the worst case, as a function of $n = \text{len}(\text{lst})$?

2. The function below builds a frequency table. Fill in the blanks.

```
def frequency(lst):
    counts = {}
    for x in lst:
        if x ----- counts:
            counts[x] = -----
        else:
            counts[x] -----
    return counts
```

What is the time complexity? How many primitive operations (assignments and comparisons) are performed in total, as a function of $n = \text{len}(\text{lst})$?

3. The function below is supposed to detect whether a list contains any duplicates in $O(n)$ time. The current implementation is $O(n^2)$. Fill in the blank with a data structure that achieves $O(n)$.

```
def has_duplicate(lst):
    seen = -----
    for x in lst:
        if x in seen:
            return True
        seen.add(x)
    return False
```

Why would using a **list** instead of your chosen structure make this $O(n^2)$?

4. The function below performs matrix–vector multiplication for an $n \times n$ matrix. Fill in the blanks and give an exact count of multiplications and additions as a function of n .

```
def matvec(M, v):
    n = len(v)
    result = [0] * n
    for i in range(-----):
        for j in range(-----):
            result[i] += -----
    return result
```

Write the Code

- Counting Inversions.** An *inversion* in a list is a pair of indices (i, j) such that $i < j$ but $\text{lst}[i] > \text{lst}[j]$. Write a function `count_inversions(lst)` that returns the total number of inversions. First, write a brute-force $O(n^2)$ solution and count the exact number of primitive comparisons it performs. Then, write an $O(n \log n)$ solution based on merge sort.
- Distinct Substring Count.** Write a function `count_distinct_substrings(s)` that returns the number of distinct substrings of a string s (including the empty string). First write a solution using a set, and analyse its time and space complexity carefully, take into account the cost of hashing a substring of length ℓ .
- Two-Sum Variants.** Consider the following three problems, each asking you to find pairs that sum to a target t in a list of n integers.
 - Unsorted, return any one pair.** Write a solution using a hash set in $O(n)$ time. How many primitive operations (lookups and insertions) does it perform in the worst case?
 - Sorted, return all pairs.** Write a two-pointer solution in $O(n)$ time. Prove that no valid pair is ever skipped.
 - Unsorted, return all pairs.** What is the best complexity you can achieve? Is it possible to do better than $O(n \log n)$? Justify your answer.

Analysis Problems

1. **Exact Operation Counts.** For each function below, give an **exact** count of the number of primitive operations performed (additions, comparisons, and assignments) as a function of n . Then give the Θ complexity.

(a)

```
def f(n):
    total = 0
    for i in range(n):
        for j in range(i, n):
            total += 1
    return total
```

(b)

```
def g(n):
    i = 1
    count = 0
    while i < n:
        count += 1
        i *= 3
    return count
```

(c)

```
def h(lst):
    n = len(lst)
    result = []
    for i in range(n):
        if all(lst[i] % lst[j] != 0 for j in range(i)):
            result.append(lst[i])
    return result
```

2. **Amortised Analysis.** A *dynamic array* starts empty and doubles its capacity whenever it is full. Each individual append may trigger a resize that copies all current elements. Yet the amortised cost per append is $O(1)$.

- Compute the **total** number of element copies performed after n append operations starting from capacity 1. Give an exact closed-form expression and simplify.
- Use the *accounting method*: assign a “credit” of c to each append such that every resize is fully paid for by stored credits. What is the minimum value of c ?
- Suppose instead the array grows by a **fixed** amount k (not doubles) each time it is full. How does the total number of copies change? What is the amortised cost per append in this case?

Chapter 14

Classes and Inheritance

Output Prediction

For each code snippet, determine what is printed. Before tracing, identify *which class's method* is actually invoked at each call, and why.

1.

```
class A:
    x = 0

    def __init__(self):
        self.x = 1

    def show(self):
        print(self.x, A.x)

class B(A):
    def __init__(self):
        A.x = 99
        self.x = 2

b = B()
a = A()
b.show()
a.show()
print(A.x)
```

2.

```
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.nxt = nxt

    def __repr__(self):
        if self.nxt is None:
            return str(self.val)
        return str(self.val) + " -> " + repr(self.nxt)
```



```

class SentinelNode(Node):
    def __repr__(self):
        return "[S]" + (" -> " + repr(self.nxt) if self.nxt
            ↪ else "")

head = SentinelNode(0, Node(1, Node(2, SentinelNode(99))))
print(head)

```

3.

```

class Counter:
    count = 0

    def __init__(self):
        Counter.count += 1
        self.id = Counter.count

    def __del__(self):
        Counter.count -= 1

c1 = Counter()
c2 = Counter()
print(Counter.count, c1.id, c2.id)
del c1
print(Counter.count)
c3 = Counter()
print(Counter.count, c3.id)

```

4.

```

class Mixin:
    def hello(self):
        return "Mixin"

class Base:
    def hello(self):
        return "Base"

    def greet(self):
        return self.hello()

class Child(Mixin, Base):
    pass

class GrandChild(Child):
    def hello(self):
        return super().hello() + "+GrandChild"

print(Child().greet())
print(GrandChild().greet())
print(GrandChild.__mro__)

```

5.

```

class Shape:
    def area(self):
        return 0

    def describe(self):
        print(f"I am a {type(self).__name__} with area {self.
        ↪ area():.1f}")

class Circle(Shape):
    def __init__(self, r):
        self.r = r

    def area(self):
        return 3.14159 * self.r ** 2

class Ring(Circle):
    def __init__(self, outer, inner):
        super().__init__(outer)
        self.inner = inner

    def area(self):
        return super().area() - Circle(self.inner).area()

shapes = [Shape(), Circle(3), Ring(5, 2)]
for s in shapes:
    s.describe()

```

Complete the Code

1. The class below implements a **stack** backed by a list. A stack is a last-in, first-out (LIFO) data structure: elements are always added and removed from the same end, called the *top*. Think of it like a stack of plates — you can only take from (or add to) the top. The three core operations are push (add to top), pop (remove and return from top), and peek (look at the top without removing it). Fill in the blanks so that these behave correctly, and so that two stacks with identical contents compare as equal.

```

class Stack:
    def __init__(self):
        self._data = []

    def push(self, val):
        -----

    def pop(self):
        if self.is_empty():
            raise IndexError("pop from empty stack")

```

```

        return -----

def peek(self):
    if self.is_empty():
        raise IndexError("peek at empty stack")
    return -----

def is_empty(self):
    return -----

def __len__(self):
    return -----

def __eq__(self, other):
    if not isinstance(other, Stack):
        return -----
    return -----

def __repr__(self):
    return f"Stack({self._data})"

s = Stack()
s.push(1); s.push(2); s.push(3)
t = Stack()
t.push(1); t.push(2); t.push(3)
print(s == t)          # True
print(s.pop())         # 3
print(s == t)          # False

```

2. The class hierarchy below models a simple expression tree. Fill in the blanks so that `eval()` correctly evaluates the expression and `__repr__` renders it with parentheses. An **expression tree with binary operations** is a way of writing a math expression where each operation combines exactly two values, and the whole expression is built step by step from these pairwise combinations.

```

class Expr:
    def eval(self):
        raise NotImplementedError

class Num(Expr):
    def __init__(self, val):
        self.val = val

    def eval(self):
        return -----

    def __repr__(self):
        return str(self.val)

```

```

class BinOp(Expr):
    def __init__(self, left, op, right):
        self.left = left
        self.op = op
        self.right = right

    def eval(self):
        l = -----
        r = -----
        if self.op == '+': return -----
        if self.op == '-': return -----
        if self.op == '*': return -----
        if self.op == '/': return -----
        raise ValueError(f"Unknown op: {self.op}")

    def __repr__(self):
        return f"({-----} {self.op} {-----})"

# (3 + 4) * (10 - 2)
e = BinOp(BinOp(Num(3), '+', Num(4)), '*', BinOp(Num(10), '-',
    ↪ , Num(2)))
print(e)                # ((3 + 4) * (10 - 2))
print(e.eval())         # 56

```

Write the Code

1. **Bank Account.** Write a class `BankAccount` with the following behaviour:

- `__init__(owner, balance=0)` — create an account with a given owner name and starting balance (default 0).
- `deposit(amount)` — add amount to the balance. Raise `ValueError` if amount is not positive.
- `withdraw(amount)` — subtract amount from the balance. Raise `ValueError` if amount is not positive or if it would make the balance negative.

Then write a subclass `SavingsAccount` that additionally charges a \$2 fee on every withdrawal, applied *after* the amount is deducted (so withdrawing \$30 reduces the balance by \$32 in total).

```

a = BankAccount('Alice', 100)
a.deposit(50)
a.withdraw(30)
print(a)                # BankAccount('Alice', 120)

s = SavingsAccount('Bob', 100)
s.withdraw(30)
print(s.balance)        # 68

```

2. **Shape Hierarchy.** Write a base class `Shape` with a method `area()` that raises `NotImplementedError`, and a method `is_larger_than(other)` that returns `True` if `self.area() > other.area()`. Then write three subclasses:

- `Rectangle(width, height)` with `area() = width * height`.
- `Circle(radius)` with `area() = 3.14159 * radius ** 2`.
- `Triangle(base, height)` with `area() = 0.5 * base * height`.

`is_larger_than` should be defined **only once** in `Shape` and work correctly for any pair of subclasses without modification.

```
r = Rectangle(4, 5)
c = Circle(3)
t = Triangle(6, 4)
print(r.area())           # 20
print(c.is_larger_than(r)) # True
print(t.is_larger_than(c)) # False
```

3. **Student Roster.** Write a class `Student` with attributes `name` and `grade` (a number from 0 to 100). Then write a class `Roster` that holds a collection of `Student` objects and supports:

- `add(student)` — add a `Student` to the roster.
- `top_student()` — return the `Student` with the highest grade. If the roster is empty, raise `ValueError`.
- `passing(threshold=50)` — return a list of all students with `grade >= threshold`, sorted by grade descending.
- `__len__` — return the number of students.

```
r = Roster()
r.add(Student('Alice', 88))
r.add(Student('Bob', 45))
r.add(Student('Carol', 95))
r.add(Student('Dan', 60))
print(len(r))           # 4
print(r.top_student().name) # Carol
print([s.name for s in r.passing()]) # ['Carol', 'Alice', '
    ↪ Dan']
```

4. **Library System.** Write a class `Book` with attributes `title`, `author`, and `available` (a boolean, default `True`). Then write a class `Library` that supports:

- `add_book(book)` — add a `Book` to the library's collection.
- `checkout(title)` — mark the book with the given title as unavailable. Raise `ValueError` if the title is not found or the book is already checked out.

- `return_book(title)` — mark the book as available again. Raise `ValueError` if the title is not found or the book was not checked out.
- `search_by_author(author)` — return a list of all books (available or not) by the given author.
- `available_books()` — return a list of all currently available books.

```
lib = Library()
lib.add_book(Book('Dune', 'Herbert'))
lib.add_book(Book('Foundation', 'Asimov'))
lib.add_book(Book('I Robot', 'Asimov'))

lib.checkout('Dune')
print(len(lib.available_books()))           # 2
print(lib.available_books()[0].title)       # Foundation
print(len(lib.search_by_author('Asimov'))) # 2

try:
    lib.checkout('Dune')    # already checked out
except ValueError as e:
    print(e)
```

Conceptual Problems

1. **Instance, Class, and Static Methods.** A colleague writes the following class and claims it correctly tracks all instances ever created:

```
class Widget:
    registry = []

    def __init__(self, name):
        self.name = name
        self.registry.append(self)

    def count(self):
        return len(self.registry)

    @staticmethod
    def reset():
        Widget.registry = []
```

- (a) Under what conditions does `self.registry.append(self)` behave differently from `Widget.registry.append(self)`? Construct a concrete example where the two diverge.
- (b) Should `count` be an instance method, a class method, or a static method? Rewrite it appropriately and justify.

- (c) `reset()` replaces `Widget.registry` with a new list. Is there a subtle bug if a subclass of `Widget` has already created instances? Explain and fix it.

Chapter 15

File Handling

Output Prediction

Predict the output or fix the errors for each of the following programs. Assume that the files mentioned in the comments already exist with the shown contents.

1.

```
# data.txt contains:
# apple
# banana
# cherry

handle = open("data.txt", "r")
print(handle.readline())
print(handle.readline())
handle.close()
```

2.

```
# nums.txt contains:
# 10
# 20
# 30

with open("nums.txt", "r") as f:
    a = f.read()
    print(a)
```

3.

```
# poem.txt contains:
# red
# blue
# green

f = open("poem.txt", "r")
first = f.readline()
rest = f.readlines()
print(first)
print(rest)
```



```
f.close()
```

4.

```
with open("log.txt", "w") as f:
    f.write("Start\n")
    f.write("End")

with open("log.txt", "r") as f:
    print(f.read())
```

5.

```
with open("story.txt", "w") as f:
    f.write("Hi")

with open("story.txt", "a") as f:
    f.write(" there")

with open("story.txt", "r") as f:
    print(f.read())
```

Complete the Code

1. **Write a Numbered Copy of a File.** Complete the function so that it reads every line from `source_file` and writes it to `target_file` with line numbers such as 1: first line.

```
def numbered_copy(source_file, target_file):
    with _____(source_file, "r") as src:
        with _____(target_file, "w") as dst:
            line_no = 1
            for line in _____:
                dst._____(str(line_no) + ": " + line)
                line_no += 1
```

2. **Append the Sum of Even Numbers to a Report File.** The file `data_file` contains one integer per line. Complete the code so that it computes the sum of all even integers and appends the result to `report_file` in the form `Even Sum = 42`.

```
def append_even_sum(data_file, report_file):
    total = 0
    with _____(data_file, "r") as handle:
        for line in _____:
            x = int(line._____)
            if x % 2 == 0:
                total += x

    with open(report_file, _____) as out:
        out._____("Even Sum = " + str(total) + "\n")
```

3. **Copy Only Non-Empty Lines.** Complete the code so that it copies all non-empty lines from `input_file` to `output_file`.

```
def copy_non_empty(input_file, output_file):  
    with _____(input_file, _____) as fin:  
        with _____(output_file, _____) as fout:  
            for line in _____:  
                if line._____() != "":  
                    fout._____ (line)
```

Word Problems

1. **Marks Report.** A file contains one student's marks per line. Write a program that reads all marks from the file and prints the highest mark, lowest mark, and average mark.
2. **Even and Odd Split.** A file contains one integer per line. Write a program that reads the numbers and stores all even numbers in `even.txt` and all odd numbers in `odd.txt`.
3. **Word Count in a File.** A text file contains multiple lines of English text. Write a program to count the total number of words in the file.
4. **Search for a Name.** A file contains one name per line. Given a target name as input, write a program to check whether the name is present in the file and print Found or Not Found.
5. **Copy Without Blank Lines.** Write a program that reads a text file and creates another file containing the same contents, but with all blank lines removed.
6. **Daily Expense Summary.** A file contains daily expenses, one amount per line. Write a program that reads the file and prints the total expense, number of entries, and average daily expense.

Chapter 16

Try and Except

Output Prediction

Predict the output or fix the errors for each of the following programs.

1.

```
try:
    x = int("15")
    print(x + 5)
except:
    print("Error")
```

2.

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Done")
```

3.

```
try:
    nums = [4, 5, 6]
    print(nums[3])
except IndexError:
    print("Bad index")
else:
    print("No error")
```

4.

```
try:
    value = int("two")
except ValueError:
    print("Value error")
except:
    print("Some other error")
finally:
    print("Finished")
```

5.

```
def f(x):
    if x < 0:
        raise ValueError("negative")
    return x * 2

try:
    print(f(-3))
except ValueError:
    print("Invalid input")
```

Complete the Code

1. **Safe Division.** Complete the code so that it prints a message instead of crashing when division by zero occurs.

```
def safe_divide(a, b):
    try:
        print(a / b)
    except _____:
        print("Division by zero is not allowed")
```

2. **Safe Integer Input.** Complete the program so that it handles non-integer input.

```
try:
    x = _____(input("Enter an integer: "))
    print("You entered", x)
except _____:
    print("Please enter a valid integer")
```

3. **Open a File Safely.** Complete the function so that it returns the contents of the file if it exists, otherwise it returns "File not found".

```
def safe_read(filename):
    try:
        with open(filename, "r") as handle:
            return handle._____()
    except _____:
        return "File not found"
```

Word Problems

1. **Robust Calculator.** Write a program that reads two numbers and an operation (+, -, *, /). It should handle invalid numbers and division by zero using exception handling.

2. **Safe List Access.** Given a list and an index entered by the user, write a program that prints the element at that index. If the index is invalid, print a suitable error message instead of crashing.
3. **Reading a Missing File.** Write a program that asks the user for a filename and prints its contents. If the file does not exist, print an error message using `FileNotFoundException`.
4. **Validated Age Input.** Write a program that reads a person's age. If the input is not an integer, handle the exception. If the age is negative, raise a `ValueError` with an appropriate message.
5. **Frequency Table with Exception Handling.** Given a list, build a dictionary storing the frequency of each element. First solve it using try-except when the key is not yet present.
6. **Retry Until Correct Input.** Write a program that keeps asking the user to enter an integer until a valid integer is entered. Use try-except inside a loop.
7. **Average of File Numbers with Bad Lines.** A file contains one value per line, but some lines may contain invalid text. Write a program that ignores invalid lines using exception handling and computes the average of all valid numbers.